

DeepAgents：开箱即用的 skill、上下文压缩等 middleware 已付费

原创 神说要有光 神光的幸福生活 2026年5月23日 22:45 山东

我们学了 LangChain、LangGraph，可以基于它们实现各种 Agent。

但如果想做一个复杂的 Agent，全部从头自己实现还是比较麻烦。

有没有基于 LangGraph 再封装一层，也就是半成品的 Agent 框架呢？

有的，就是 DeepAgents 。

LangChain 是给你一堆 AI 开发积木，LangGraph 是搭建复杂工作流的底层蓝图，那 DeepAgents 就是提前搭好主体结构的半成品房子。

底层依赖 LangGraph 的状态管理 (state)、循环路由、持久化执行能力 (checkpointing)，上层直接内置了任务规划、长期记忆、子 Agent 调度、上下文压缩等核心能力。

它最大的优势，就是大幅降低复杂 Agent 的开发门槛。

原生 LangGraph 适合极致自定义、底层深度开发

DeepAgents 适合快速落地复杂 Agent 应用，比如深度调研、代码开发、多步骤业务执行、多智能体协作等场景。

它帮我们跳过重复的底层基建，直接聚焦 Agent 的业务逻辑与能力迭代，是 LangGraph 生态里面向生产落地的高阶封装方案。

接下来我们就来学一下 DeepAgents：

创建项目：

```
mkdir deepagents-test  
cd deepagents-test  
npm init -y
```

安装依赖：

```
pnpm install deepagents langchain @langchain/langgraph @langchain/openai zod dotenv
```

创建 .env

```
OPENAI_BASE_URL=https://dashscope.aliyuncs.com/compatible-mode/v1  
MODEL_NAME=qwen-plus  
OPENAI_API_KEY=sk-xxx  
  
# 用于身份验证，实现链路上报  
LANGCHAIN_API_KEY=xxx  
# 指定LangSmith中的项目，追踪结果会归类到该项目下  
LANGCHAIN_PROJECT=deepagents-test  
# 开启LangSmith追踪功能  
LANGCHAIN_TRACING_V2=true
```

然后先来试一下 middleware，这个是 langchain 的功能：

创建 src/middleware-test.mjs

```
import "dotenv/config";
import { z } from "zod";
import { ChatOpenAI } from "@langchain/openai";
import {
  createAgent,
  createMiddleware,
  HumanMessage,
  AIMessage,
} from "langchain";

// --- 自定义 Middleware ---

/** 日志 + 模型调用次数统计 */
const loggingMiddleware = createMiddleware({
  name: "LoggingMiddleware",
  stateSchema: z.object({
    modelCallCount: z.number().default(0),
  }),
  beforeAgent: (state) => {
    console.log(`\n[Logging] agent 开始, 消息数:`, state.messages.length);
  },
  beforeModel: (state) => {
    console.log(
      `[Logging] 即将调用模型, 当前消息数: ${state.messages.length}, 已调用: ${state.modelCallCount}`
    );
  },
  afterModel: (state) => {
    const last = state.messages.at(-1);
    const preview =
      typeof last?.content === "string"
        ? last.content.slice(0, 80)
        : JSON.stringify(last?.content)?.slice(0, 80);
    console.log(`[Logging] 模型返回: ${preview}...`);
    return { modelCallCount: state.modelCallCount + 1 };
  },
  afterAgent: (state) => {
    console.log(
      `[Logging] agent 结束, 累计模型调用: ${state.modelCallCount} 次\n`
    );
  },
});

/** 在每次模型调用前追加 system 上下文 */
const addContextMiddleware = createMiddleware({
```

```

name: "AddContextMiddleware",
wrapModelCall: async (request, handler) => {
  console.log("[AddContext] 注入额外 system 上下文");
  return handler({
    ...request,
    systemMessage: request.systemMessage.concat(
      "\n\n 请用一句话简洁回答。"
    ),
  });
},
});

/** 拦截敏感词, 直接结束 agent */
const blockedContentMiddleware = createMiddleware({
name: "BlockedContentMiddleware",
beforeModel: {
  canJumpTo: ["end"],
  hook: (state) => {
    const last = state.messages.at(-1);
    const text =
      typeof last?.content === "string" ? last.content : String(last?.content ?? "")
    if (text.includes("BLOCKED")) {
      console.log("[Blocked] 检测到 BLOCKED, 短路结束");
      return {
        messages: [new AIMessage("该请求已被 middleware 拦截, 无法处理。")],
        jumpTo: "end",
      };
    }
  },
},
});

// --- Agent ---

const model = new ChatOpenAI({
model: process.env.MODEL_NAME,
apiKey: process.env.OPENAI_API_KEY,
configuration: {
  baseUrl: process.env.OPENAI_BASE_URL,
},
temperature: 0,
});

const agent = createAgent({
  model,
  tools: [],
  systemPrompt: "你是一个助手。",
});

```

```

middleware: [
    loggingMiddleware,
    addContextMiddleware,
    blockedContentMiddleware,
],
});

for (const text of [
    "用中文说: middleware 是什么? ",
    "这句话包含 BLOCKED 关键词",
]) {
    console.log("\n用户:", text);
    const { messages, modelCallCount } = await agent.invoke({
        messages: [new HumanMessage(text)],
    });
    console.log("回复:", messages.at(-1)?.content);
    console.log("modelCallCount:", modelCallCount);
}

```

createAgent 这个 api 提供了 middleware 的扩展机制:

可以在 agent 运行前后、model 调用前后加一些逻辑, 以及控制 model 要不要调用, 可以提前结束流程

神光的编程秘籍

此外，中间件还可以扩展 tool，以及 wrapToolCall

创建 src/middleware-test2.mjs

```
import "dotenv/config";
import { Command } from "@langchain/langgraph";
import { z } from "zod";
import { ChatOpenAI } from "@langchain/openai";
import {
  createAgent,
  createMiddleware,
  HumanMessage,
  ToolMessage,
  tool,
} from "langchain";

const getCurrentTime = tool(() => new Date().toISOString(), {
  name: "get_current_time",
  description: "返回当前 UTC 时间的 ISO 8601 字符串",
  schema: z.object({}),
});

/** 通过 middleware 注册工具，并用 wrapToolCall 包装执行 */
const extendedToolsMiddleware = createMiddleware({
  name: "ExtendedToolsMiddleware",
  stateSchema: z.object({
    toolInvocationCount: z.number().default(0),
  }),
  tools: [getCurrentTime],
  wrapToolCall: async (request, handler) => {
    const toolName = request.tool?.name ?? request.toolCall.name;
    console.log(
```

```

    `[Tools] 即将执行: ${toolName}`,
    "args:",
    request.toolCall.args ?? {}
  );
  const result = await handler(request);
  if (!ToolMessage.isInstance(result)) return result;

  const wrapped = new ToolMessage({
    content: `${result.content}\n[wrapToolCall] 已由 ExtendedToolsMiddleware 包装`,
    tool_call_id: result.tool_call_id,
    name: result.name,
  });
  console.log(
    `[Tools] 执行完成: ${toolName}`,
    typeof wrapped.content === "string"
      ? wrapped.content.slice(0, 120)
      : wrapped
  );
  return new Command({
    update: {
      toolInvocationCount: request.state.toolInvocationCount + 1,
      messages: [wrapped],
    },
  });
},
afterAgent: (state) => {
  console.log(
    `[Tools] agent 结束, middleware 统计工具调用: ${state.toolInvocationCount} 次`
  );
},
});

const model = new ChatOpenAI({
  model: process.env.MODEL_NAME,
  apiKey: process.env.OPENAI_API_KEY,
  configuration: {
    baseUrl: process.env.OPENAI_BASE_URL,
  },
  temperature: 0,
});

const agent = createAgent({
  model,
  tools: [],
  systemPrompt:
    "你是一个助手。",
  middleware: [extendedToolsMiddleware],
});

```

```
for (const text of [
  "给我当前时间",
]) {
  console.log("\n用户:", text);
  const { messages, toolInvocationCount } = await agent.invoke({
    messages: [new HumanMessage(text)],
  });
  console.log("回复:", messages.at(-1)?.content);
  console.log("toolInvocationCount:", toolInvocationCount);
}
```

神光的编程秘籍

这样我们就通过中间件给 agent 扩展了 tools 并且修改了 tool call 返回的结果

deepagents 里就有很多现成的中间件可以用：

先试一下 `FilesystemMiddleware`，这个中间件可以指定一个 backend 作为文件系统，然后提供了读写、修改、搜索文件的命令。

创建 `src/deepagents/filesystem-agent.mjs`

```
import "dotenv/config";
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { ChatOpenAI } from "@langchain/openai";
import { createAgent, HumanMessage } from "langchain";
import { createFilesystemMiddleware, FilesystemBackend } from "deepagents";

const workspaceDir = path.join(
  path.dirname(fileURLToPath(import.meta.url)),
  "workspace"
);

/** 先匹配先生效；未命中任何规则则默认允许 */
const permissions = [
  { operations: ["read"], paths: ["/secret.txt"], mode: "deny" },
  { operations: ["write"], paths: ["/todo.md"], mode: "allow" },
  { operations: ["write"], paths: ["/**"], mode: "deny" },
];

fs.rmSync(workspaceDir, { recursive: true, force: true });
fs.mkdirSync(workspaceDir);
fs.writeFileSync(path.join(workspaceDir, "secret.txt"), "机密：不得读取", "utf8");
```

```

const model = new ChatOpenAI({
  model: process.env.MODEL_NAME,
  apiKey: process.env.OPENAI_API_KEY,
  configuration: { baseURL: process.env.OPENAI_BASE_URL },
  temperature: 0,
});

const agent = createAgent({
  model,
  tools: [],
  systemPrompt:
    "工作区根路径为 /。用 ls、read_file、write_file、edit_file 操作文件，路径以 / 开头。中文回答",
  middleware: [
    createFilesystemMiddleware({
      backend: new FilesystemBackend({ rootDir: workspaceDir, virtualMode: true }),
      permissions,
    }),
  ],
});

console.log("工作区:", workspaceDir);
console.log("权限:", JSON.stringify(permissions, null, 2));

asyncfunction run(label, prompt) {
  console.log(`\n=== ${label} ===\n`, prompt, "\n");
  const { messages } = await agent.invoke(
    { messages: [new HumanMessage(prompt)] },
    { recursionLimit: 20 }
  );
  for (const m of messages) {
    for (const t of m.tool_calls ?? []) console.log("→", t.name);
  }
  console.log("回复:", messages.at(-1)?.content);
}

asyncfunction expectDenied(label, prompt) {
  console.log(`\n=== ${label} (预期拒绝) ===\n`, prompt, "\n");
  try {
    await agent.invoke({ messages: [new HumanMessage(prompt)] }, { recursionLimit: 5 });
    console.log("未触发拒绝 (异常)");
  } catch (e) {
    const msg = e.cause?.message ?? e.message;
    console.log("x", msg);
  }
}

await run(

```

```
"允许的操作",  
"write_file 创建 /todo.md (三条待办), edit_file 把第一条标为完成, ls /, 一句话总结."  
);  
  
await expectDenied("禁止读", "只调用 read_file, 路径 /secret.txt。");  
await expectDenied("禁止写", "只调用 write_file, 路径 /hack.txt, 内容 test。");
```

神光的编程秘籍

只要加上 deepagents 这个 FileSystem 中间件, agent 就有了一个文件系统, 并且有了读写搜索文件的各种 tool, 还做了权限控制。

超级方便, 不用自己写!

大家应该都用过 skill, 如果我们的 Agent 也要支持 skill 呢?

直接用 deepagents 的 Skill sMiddleware

创建 src/deepagents/skills-agent.mjs

```
import "dotenv/config";  
import { existsSync, mkdirSync } from "node:fs";  
import { ChatOpenAI } from "@langchain/openai";  
import { createAgent, HumanMessage } from "langchain";  
import {  
  LocalShellBackend,  
  createFileSystemMiddleware,  
  createSkillsMiddleware,  
} from "deepagents";
```

```

const skills = "/.agents/skills/";
const output = "src/deepagents/output/deepagents-skills-flow.excalidraw";

if (!existsSync(".agents/skills/excalidraw-diagram-generator/SKILL.md")) {
  throw new Error(
    "未找到 excalidraw-diagram-generator, 请先: npx skills add github/awesome-copilot --:
  );
}

mkdirSync("src/deepagents/output", { recursive: true });

const model = new ChatOpenAI({
  model: process.env.MODEL_NAME,
  apiKey: process.env.OPENAI_API_KEY,
  configuration: { baseUrl: process.env.OPENAI_BASE_URL },
  temperature: 0,
  streaming: true,
});

const backend = await LocalShellBackend.create({
  rootDir: ".",
  virtualMode: true,
  inheritEnv: true,
});

const agent = createAgent({
  model,
  tools: [],
  systemPrompt: "按 skills 库完成任务, 需要时 read_file 对应 SKILL.md。中文回答。",
  middleware: [
    createSkillsMiddleware({ backend, sources: [skills] }),
    createFilesystemMiddleware({ backend }),
  ],
});

const prompt = [
  "画一张流程图, 描述本项目的 skills-agent 工作流:",
  "用户 Prompt → createAgent → createSkillsMiddleware → createFilesystemMiddleware → 模型",
  "保存为 ${output}。要求:",
  "- 顶部大标题 + 副标题",
  "- 每个主节点 numbered (①②...) 且框内 2~3 行中文说明",
  "- 右侧一列「说明: ...」补充细节",
  "- 箭头上标注阶段名 (如 invoke、wrapModelCall)",
  "- 底部图例 (颜色含义 + 如何运行 demo)",
].join("\n");

```

```
console.log("用户:", prompt);

function chunkText(chunk) {
  if (!chunk?.content) return "";
  if (typeof chunk.content === "string") return chunk.content;
  if (Array.isArray(chunk.content)) {
    return chunk.content
      .map((p) => (typeof p === "string" ? p : (p?.text ?? "")))
      .join("");
  }
  return "";
}

const stream = await agent.streamEvents(
  { messages: [new HumanMessage(prompt)] },
  { recursionLimit: 100 }
);

let skillsMetadata;
console.log("\n--- 流式输出 ---\n");

try {
  forawait (const event of stream) {
    if (event.event === "on_chat_model_stream") {
      const text = chunkText(event.data?.chunk);
      if (text) process.stdout.write(text);
    }
    if (event.event === "on_tool_start") {
      const name = event.name?.split("/").pop() ?? event.name;
      process.stdout.write(`\n\n→ ${name}\n\n`);
    }
    if (event.event === "on_chain_end" && event.data?.output?.skillsMetadata) {
      skillsMetadata = event.data.output.skillsMetadata;
    }
  }
} catch (e) {
  console.error("\n\n[错误]", e.cause?.message ?? e.message);
  throw e;
}

console.log("\n");
console.log("skills:", skillsMetadata?.map((s) => s.name));
if (existsSync(output)) {
  console.log("图表:", output);
  console.log("打开: https://excalidraw.com → Open → 选择该文件");
} else {
  console.log("未生成:", output);
}
```

```
}
```

```
await backend.close();
```

从 <https://www.skills.sh/> 查找 skill

神光的编程秘籍

这样，我们的 Agent 就支持 skill 了！

再来试一下 deepagents 其他中间件：

SubAgentMiddleware 这个是创建多 Agent 用的

创建 src/deepagents/subagent-agent.mjs

```
import "dotenv/config";
import { z } from "zod";
import { ChatOpenAI } from "@langchain/openai";
import { createAgent, HumanMessage, tool } from "langchain";
import { createSubAgentMiddleware } from "deepagents";

/** 四则运算 */
const calc = tool(
  ({ a, b, op }) => {
    const ops = {
      add: a + b,
      subtract: a - b,
      multiply: a * b,
      divide: b === 0 ? NaN : a / b,
    };
    const result = ops[op];
    if (Number.isNaN(result)) {
      return JSON.stringify({ error: "除数不能为 0" });
    }
    const symbols = { add: "+", subtract: "-", multiply: "x", divide: "÷" };
    return JSON.stringify({
      expression: `${a} ${symbols[op]} ${b}`,
      result,
    });
  },
  {
    name: "calc",
    description: "计算两个数的加减乘除",
    schema: z.object({
      a: z.number().describe("左操作数"),
      b: z.number().describe("右操作数"),
      op: z.enum(["add", "subtract", "multiply", "divide"]).describe("运算类型"),
    }),
  }
);

/** 平均分: 总数 ÷ 份数 */
const divideEvenly = tool(
  ({ total, parts }) => {
    if (parts <= 0) {
      return JSON.stringify({ error: "份数须大于 0" });
    }
    const each = total / parts;
    const exact = Number.isInteger(each);
```

```

returnJSON.stringify({
  total,
  parts,
  each,
  exact,
  note: exact
    ? `每人 ${each} (整除)`
    : `每人 ${each} (不能整除, 应用题可说明余数)`,
});
},
{
  name: "divide_evenly",
  description: "把总数平均分成若干份, 求每份多少",
  schema: z.object({
    total: z.number().nonnegative().describe("总数"),
    parts: z.number().int().positive().describe("分成几份"),
  }),
}
);

/** 按模板生成同类练习题 (只改数字) */
const makeSimilarProblem = tool(
  ({ template, seed }) => {
    const n = (seed % 7) + 3;
    const problems = {
      divide_then_add: {
        stem: `小红有 ${n * 6} 张贴纸, 平均分给 ${n} 个小组, 又买了 2 包每包 ${n + 2} 张的。每`
        hint: "先平均分, 再加上后来买的, 注意单位是「每个小组」",
      },
      share_candy: {
        stem: `小刚有 ${n * 4} 块糖, 要分给 ${n} 位同学, 妈妈又买了 3 袋每袋 ${n} 块的。每位同学`
        hint: "与分糖题类似: 先平分, 再加上新增",
      },
      group_buy: {
        stem: `班里有 ${n} 个小组, 每组先分到 ${n * 5} 支铅笔, 老师又补了 2 盒每盒 ${n + 1} 支。`
        hint: "先算每组原有, 再加上后来补的",
      },
    };
    const picked = problems[template] ?? problems.share_candy;
    returnJSON.stringify({ template, ...picked });
  },
  {
    name: "make_similar_problem",
    description:
      "生成一道同类应用题。template: divide_then_add | share_candy | group_buy",
    schema: z.object({
      template: z

```



```

        .enum(["divide_then_add", "share_candy", "group_buy"])
        .describe("题目模板"),
        seed: z.number().int().describe("随机种子，用于变换数字"),
    }),
}
);

const model = new ChatOpenAI({
  model: process.env.MODEL_NAME,
  apiKey: process.env.OPENAI_API_KEY,
  configuration: { baseUrl: process.env.OPENAI_BASE_URL },
  temperature: 0,
  streaming: true,
});

const subagents = [
  {
    name: "math-solver",
    description:
      "解小学应用题：用 calc、divide_evenly 列式计算，给出最终答案与算式。有具体数字时先用此 Agent",
    systemPrompt: [
      "你是解题子 Agent。",
      "必须用 calc、divide_evenly 完成计算，不要心算。",
      "输出：题目理解、分步算式、最终答案（带单位「块/人」等）。",
    ].join("\n"),
    tools: [calc, divideEvenly],
  },
  {
    name: "kid-tutor",
    description:
      "把 math-solver 的解法讲给家长听，方便辅导孩子。description 里会有完整解题过程。",
    systemPrompt: [
      "你是辅导讲解子 Agent，面向小学生家长。",
      "根据 description 中的解题过程，用短句、比喻或分步提问方式讲解（不要堆公式）。",
      "说明：先想什么、再算什么、怎么检查答案。不使用工具。",
    ].join("\n"),
    tools: [],
  },
  {
    name: "practice-maker",
    description:
      "出 2 道同类练习题。用 make_similar_problem 生成题干，可换不同 template 或 seed。",
    systemPrompt: [
      "你是出题子 Agent。",
      "调用 make_similar_problem 至少 2 次（不同 template 或不同 seed），",
      "每道题给出：题干、解题提示（一句话）。",
    ].join("\n"),
  },
];

```

```

    tools: [makeSimilarProblem],
  },
];

const agent = createAgent({
  model,
  tools: [],
  systemPrompt: [
    "你是小学数学辅导主 Agent，通过 task 委派子 Agent，自己不解题、不讲题、不出题。",
    "按顺序：① math-solver ② kid-tutor (把 solver 完整过程写进 description) ③ practice-maker",
    "最后向家长汇总：答案、辅导要点、两道练习题。中文。",
  ].join("\n"),
  middleware: [
    createSubAgentMiddleware({
      defaultModel: model,
      subagents,
      generalPurposeAgent: false,
    }),
  ],
});

const prompt = [
  "孩子遇到这道题：",
  "「小明有 24 块糖，平均分给 6 个同学；",
  "妈妈又买了 3 包糖，每包 5 块。每个同学现在一共有多少块？」",
  "请先 math-solver 解题，再 kid-tutor 教家长怎么讲，",
  "最后 practice-maker 出 2 道类似练习题，并汇总给我。",
].join("");

function chunkText(chunk) {
  if (!chunk?.content) return "";
  if (typeof chunk.content === "string") return chunk.content;
  if (Array.isArray(chunk.content)) {
    return chunk.content
      .map((p) => (typeof p === "string" ? p : (p?.text ?? "")))
      .join("");
  }
  return "";
}

console.log("场景：小学应用题辅导（解题 → 讲题 → 出题）");
console.log("子 Agent:");
console.log("  math-solver    → calc, divide_evenly");
console.log("  kid-tutor       → （讲解，无工具）");
console.log("  practice-maker  → make_similar_problem");
console.log();

```

```
console.log("用户:", prompt, "\n");
console.log("--- 流式输出 ---\n");

const stream = await agent.streamEvents(
  { messages: [new HumanMessage(prompt)] },
  { recursionLimit: 60 }
);

try {
  forawait (const event of stream) {
    if (event.event === "on_chat_model_stream") {
      const t = chunkText(event.data?.chunk);
      if (t) process.stdout.write(t);
    }
    if (event.event === "on_tool_start") {
      const name = event.name?.split("/").pop() ?? event.name;
      process.stdout.write(`\n\n→ ${name}\n\n`);
    }
  }
} catch (e) {
  console.error("\n\n[错误]", e.cause?.message ?? e.message);
  throw e;
}

console.log("\n");
```

此外，长期记忆也是 Agent 必备的功能，deepagents 提供了 MemoryMiddleware

可以把记忆存储在 markdown 文件里，可以读取、更新，持久化存储

创建 src/deepagents/memory-agent.mjs

```
import "dotenv/config";
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { ChatOpenAI } from "@langchain/openai";
import { createAgent, HumanMessage } from "langchain";
import {
  createFileSystemMiddleware,
  createMemoryMiddleware,
  FileSystemBackend,
} from "deepagents";

const __dirname = path.dirname(fileURLToPath(import.meta.url));
const workspaceDir = path.join(__dirname, "workspace-memory");
const projectMemoryPath = "/AGENTS.md";
const preferencesMemoryPath = "/memory/preferences.md";

const model = new ChatOpenAI({
  model: process.env.MODEL_NAME,
  apiKey: process.env.OPENAI_API_KEY,
  configuration: { baseURL: process.env.OPENAI_BASE_URL },
  temperature: 0,
});

const backend = new FileSystemBackend({
  rootDir: workspaceDir,
  virtualMode: true,
});

const agent = createAgent({
  model,
  tools: [],
  systemPrompt: [
    "你是项目助手。工作区根路径为 /，可用 ls、read_file、write_file、edit_file。",
    "根据 <agent_memory> 回答；用户要求记住时，必须立刻 edit_file，且按类型写入对应文件：",
    `- ${projectMemoryPath}: 项目说明、技术栈、架构、仓库约定等`,
    `- ${preferencesMemoryPath}: 用户个人偏好（语言、包管理器、回答风格等）`,
    "不要混写：项目事实不要写入 preferences，个人偏好不要写入 AGENTS.md。",
  ].join("\n"),
  middleware: [
```

```

    createFileSystemMiddleware({ backend }),
    createMemoryMiddleware({
      backend,
      sources: [projectMemoryPath, preferencesMemoryPath],
    }),
  ],
});

const prompts = [
  "根据记忆，这个项目是做什么的？只答一句。",
  `请记住：我常用的包管理器是 pnpm。`,
  `请记住：本仓库主入口脚本是 src/deepagents/memory-agent.mjs。`,
  "我常用什么包管理器？本 demo 主入口脚本路径是什么？各用一行回答。",
];

let messages = [];

for (const prompt of prompts) {
  console.log("\n用户:", prompt);
  ({ messages } = await agent.invoke(
    { messages: [...messages, new HumanMessage(prompt)] },
    { recursionLimit: 30 }
  ));
  console.log("回复:", messages.at(-1)?.content);
}

for (const p of [projectMemoryPath, preferencesMemoryPath]) {
  const file = path.join(workspaceDir, p.replace(/\^\/\//, ""));
  console.log(`\n--- ${p} ---\n`, fs.readFileSync(file, "utf8"));
}

```

这样，agent 就可以从 md 文件读取长期记忆，并且你让他记住的信息也会更新到 md 文件里。

最后，还有一个 SummarizationMiddleware 的中间件，它的作用是如果当前对话上下文长度超过预设阈值，就自动对历史对话进行摘要压缩，剔除冗余信息，只保留关键上下文摘要，再传入大模型进行后续续写 / 问答。

这样可以控制 Token 消耗、避免上下文溢出，同时保证核心对话语义不丢失。

我们自己做这个压缩还是比较麻烦的，这个 middleware 可以帮我们完成。

创建 src/deepagents/summarization-agent.mjs

```
import "dotenv/config";
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { ChatOpenAI } from "@langchain/openai";
import { createAgent, HumanMessage } from "langchain";
import { createSummarizationMiddleware, FilesystemBackend } from "deepagents";

const __dirname = path.dirname(fileURLToPath(import.meta.url));
const workspaceDir = path.join(__dirname, "workspace-summarization");
const historyPathPrefix = "/conversation_history";

const summaryPrompt = `你是对话摘要助手。请用中文总结以下对话，包含：
1. 讨论的主要话题
2. 达成的关键结论或决定
3. 继续对话所需的重要上下文

保持简洁，不要罗列无细节。

待摘要的对话：
{conversation}

摘要：`;

fs.rmSync(workspaceDir, { recursive: true, force: true });
fs.mkdirSync(workspaceDir, { recursive: true });

const model = new ChatOpenAI({
  model: process.env.MODEL_NAME,
```

```

apiKey: process.env.OPENAI_API_KEY,
configuration: { baseUrl: process.env.OPENAI_BASE_URL },
temperature: 0,
});

const backend = new FileSystemBackend({
  rootDir: workspaceDir,
  virtualMode: true,
});

const agent = createAgent({
  model,
  tools: [],
  systemPrompt:
    "你是会话助手。记住用户提到的关键事实，中文简短回答。若看到「此前对话摘要」，请据此继续对话。",
  middleware: [
    createSummarizationMiddleware({
      model,
      backend,
      historyPathPrefix,
      summaryPrompt,
      // 低阈值便于 demo 触发摘要；生产环境可省略 trigger/keep，由模型 profile 自动推断
      trigger: { type: "messages", value: 8 },
      keep: { type: "messages", value: 4 },
    }),
  ],
});

const prompts = [
  "请记住：我的宠物猫叫小橘。",
  "请记住：我住在北京。",
  "请记住：我喜欢喝拿铁。",
  "请记住：我的生日是 5 月 1 日。",
  "根据我们聊过的内容，我的猫叫什么、住哪、喜欢喝什么、生日是哪天？每项一行。",
];

const historyDir = path.join(workspaceDir, historyPathPrefix.replace(/^\//, ""));

function listHistoryFiles() {
  if (!fs.existsSync(historyDir)) return [];
  return fs.readdirSync(historyDir);
}

let messages = [];
let knownHistory = new Set(listHistoryFiles());

for (const prompt of prompts) {
  console.log("\n用户:", prompt);

```

```

    ({ messages } = await agent.invoke(
      { messages: [...messages, new HumanMessage(prompt)] },
      { recursionLimit: 30 }
    ));

    console.log("回复:", messages.at(-1)?.content);
    console.log("当前消息数:", messages.length);

    const historyFiles = listHistoryFiles();
    for (const file of historyFiles) {
      if (!knownHistory.has(file)) {
        knownHistory.add(file);
        console.log("已触发摘要, 历史已写入:", `${historyPathPrefix}/${file}`);
      }
    }
  }

  if (knownHistory.size > 0) {
    for (const file of knownHistory) {
      const filePath = path.join(historyDir, file);
      console.log(`\n--- ${historyPathPrefix}/${file} ---\n`, fs.readFileSync(filePath, 'utf-8'));
    }
  } else {
    console.log("\n未生成 conversation_history (可能未触发摘要阈值)");
  }
}

```

我们按照条数来摘要，达到 8 条触发摘要，保留 4 条，前面的变成摘要

这样摘要后聊的再多也只保留最新的几条，更前面的都变成摘要了：

有三种触发摘要的方式：

这个 middleware 也是很有用的。

至此，我们就把 deepagents 提供的 middleware 过了一遍。

代码上传了课程仓库：<https://github.com/QuarkGluonPlasma/ai-agent-course-code>

总结

DeepAgents 提供了很多开箱即用的功能，做 Agent 可以直接用。

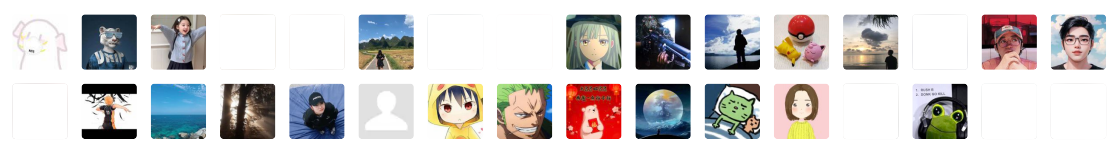
这节我们学了它的各种 middleware。

middleware 是 createAgent 提供的机制，可以在大模型调用前后、tool 调用前后加一些逻辑，修改 state、参数、扩展 tool 等。

DeepAgents 提供了 skill、上下文压缩、长期记忆（md）、文件系统、subagent 的 middleware，直接用很方便。

当然，DeepAgents 不只有中间件，下节我们继续来学习其他功能。

2774 人付费



转型 Agent 全栈工程师：企业级知识库项目 · 目录

上一篇

LangSmith 全链路观测：从 Agent 调试到 RAG 量化评估

下一篇

DeepAgents 实战：多 Agent 架构的深度调研助手

留言 5

写留言





Shichao 上海 6月5日

有了deepAgents后，是不是就没有人裸写langgraph或者langchain了



神光的幸福生活 作者 6月5日

有的，这个只是 agent 的部分功能，而且也不够灵活，适合的场景才用

1



() 广东 6月4日

感觉 middleware 这个概念在 agent 中已经不太合适



xixi 北京 6月3日

最早的langchain是不是0.x的版本 然后最新的文档都是1.



LDQ 上海 5月25日

追踪好烧脑啊，想看中间件执行过程的表现，langsmith都翻烂了